

Security in Software Applications a.y. 23-24

Project Report

Ilaria Scarselli - scarselli.1918975@studenti.uniroma1.it

January 25, 2024

1 Introduction

The project aimed to enhance the provided Solidity code to meet the properties outlined in the guidelines, thereby avoiding unexpected and undesired behaviors. In pursuit of this objective, the selection of a tool for conducting automated tests became imperative to uncover any potential vulnerabilities within the code. This approach not only ensured the alignment with specified requirements but also facilitated a proactive identification and resolution of issues, contributing to the overall robustness of the project. Additionally, considerations were given to other invariants to ensure the correct functioning of the code, beyond the primary properties earlier delineated.

1.1 Testing techniques

In the context of this project, the fuzzing technique was employed to conduct the testing process of the Solidity code. Fuzzing is a software testing method that involves feeding a program with a large amount of random, invalid, or unexpected data, to discover vulnerabilities, bugs, or potential issues and weaknesses in the code, contributing to the improvement of overall code resilience and reliability. The goal of fuzz testing is to identify points of failure within a software application by subjecting it to various input variations, including edge cases and unconventional data inputs. In this case, the fuzzing technique was employed as automated testing process. Additional tests were conducted manually using the Remix IDE, deploying test contracts with manual inputs for the functions. The static analyzer of Remix also played a significant role in enhancing the code, providing warnings and errors that I could study and fix.

1.2 Tools

The project was carried out on an Ubuntu 22.04 environment within a virtual machine. The main tools used were Echidna for automated testing and Remix IDE for manually checking the compilation and gaining a deeper understanding of the actual code functionality.

Echidna is a fuzzer designed to test smart contracts on the Ethereum blockchain. This tool employs



Invariants can be implemented in the code as control functions, using `require()` or using `assert()` statements, depending on the context. Tests are initiated from the terminal using the command:

which takes the file as an argument. If the file contains multiple contracts, the specific contract with the testing functions should follow the '*echidna_functionName*' standard. Additionally, '-format text' has been added to enable proper display, due to issues on my terminal visualization.

```
testMaxGas: 8000030  
testLimit: 500000  
seqLen: 2000 # with the default option of 8 "actions" performed  
# per test , some tests passed without being  
# correctly analyzed  
  
shrinkLimit: 5000  
sender: ["0x10000", "0x20000", "0x30000"]  
allContracts: true # to test all the contracts initialized in  
# the constructor. The default is false , where  
# just a single contract is analyzed  
  
balanceAddr: 0xfffffffff  
codeSize: 0x6000  
maxValue: 100000000000000000000 # 100 ETH  
dictFreq: 0.40  
maxTimeDelay: 604800  
maxBlockDelay: 60480  
filterFunctions: []  
coverageFormats: ["txt","html","lcov"]  
workers: 8 # the default is 1. Increasing this number allows  
# the use of parallelization
```



2 Part 1 - Marriage problems

In the initial phase, the primary challenge was to ensure that the marriages were symmetrical. To achieve this, a function called `marriageSymmetry()` was created within the main contract to facilitate checks, as it was later reused in the `require` statements throughout the code. Then, I introduced the `testTaxpayer` contract within the same file to start the first testing. There are three approaches for Echidna functions: you can include them within the contract itself, create a new contract in a separate file, or create one in the same file. The creation of a contract allows for a constructor (Echidna requires a constructor without input arguments) where the instances to be tested can be initialized. In the code, several functions and getters have been added. To make the testing phase easier, a decision was made to modify the constructor of `Taxpayer` by omitting the inclusion of parents, as they are not currently used in the functions. Alternatively, they could have been kept, but they had to be manually initialized in the `testTaxpayer` constructor.

2.1 Marriage symmetry

```

39
40
41 //We require new_spouse != address(0);
42 function marry(address new_spouse) public {
43     require(!isMarried);
44     Taxpayer sp = Taxpayer(address(new_spouse));
45     require(!sp.getIsMarried());
46     spouse = address(sp);
47     isMarried = true;
48     //sp.setSpouse(address(this));
49     //require(marriageSymmetry(), "Marriage symmetry invariant violated");
50 }
51
52 function divorce() public {
53     Taxpayer sp = Taxpayer(address(spouse));
54     sp.setSpouse(address(0));
55     spouse = address(0);
56     isMarried = false;
57 }
58
59 /* Transfer part of tax allowance to own spouse */

```

```

[2024-01-24 13:31:14.86] [Worker 0] New coverage: 864 instr, 1 contracts, 1 seqs in corpus
[2024-01-24 13:31:14.98] [Worker 0] New coverage: 1003 instr, 1 contracts, 2 seqs in corpus
[2024-01-24 13:31:15.04] [Worker 0] New coverage: 1039 instr, 1 contracts, 3 seqs in corpus
[2024-01-24 13:31:15.13] [Worker 0] Test echidna_testMarriageSymmetry falsified!
[2024-01-24 13:31:15.13] INFO: Performing RPC: <EVM.Query: fetch contract 0x0000000000000000000000000000000000000000DeaDBeefEmptyBase>
[2024-01-24 13:31:15.13] ERROR: Requested RPC but it is not configured: <EVM.Query: fetch contract 0x0000000000000000000000000000000000000000DeaDBeefEmptyBase>
[2024-01-24 13:31:15.28] [Worker 0] New coverage: 1127 instr, 1 contracts, 4 seqs in corpus
[2024-01-24 13:31:17.83] [status] tests: 1/1, fuzzing: 1085/50000, values: [], cov: 1127, corpus: 4
[2024-01-24 13:31:18.18] [Worker 0] Test limit reached. Stopping.
[2024-01-24 13:31:18.18] [status] tests: 1/1, fuzzing: 1085/50000, values: [], cov: 1127, corpus: 4
echidna_testMarriageSymmetry: failed! ✖
Call sequence:
  marry(0xa329c0648769a73afac7f9381e08fb43dbea72)
  setSpouse(0x0)
Traces:
call 0x0000000000000000000000000000000000000000000000000000000000000000::getSpouse() (/home/cheza/Scarlatti/src/Taxpayer.sol:
94)
↳ (0x)

```

NOTE: Part of the code in the images was then further modified while dealing with subsequent errors and invariants. I added the getter for `isMarried` and the initial `require` conditions, modifying the `marry()` function, and subsequently also the `setSpouse()` function, which I further specialized by creating `resetSpouse()`, called by `divorce()`. As a result, `divorce()` will no longer invoke `setSpouse()`.

```

// Added function to verify marriage symmetry
function marriageSymmetry() view public returns (bool) {
    // Checks the symmetry iff the contract is married
    if (isMarried) {
        // Obtains spouse's contract
        Taxpayer sp = Taxpayer(address(spouse));
        // Checks if the spouse is married to the correct address
        return sp.getSpouse() == address(this) && sp.getIsMarried();
    }
    // If the contract is not married, the property is considered valid

```




If Alice is not married, then it is correct for the address contained in spouse to be zero. Therefore, we only check this condition when isMarried is true.

```
//We require new_spouse != address(0);
function marry(address new_spouse) public {
    // Added checks to see if the two contracts are not already married and
    if the spouse address is valid
    require(!isMarried && new_spouse != address(0));
    Taxpayer sp = Taxpayer(address(new_spouse));
    require(!sp.getIsMarried());
    spouse = address(new_spouse);
    isMarried = true;
    sp.setSpouse(address(this));
    require(marriageSymmetry());
}
```

```
[2024-01-24 16:43:39.24] [Worker 0] New coverage: 1513 instr, 1 contracts, 3 seqs in corpus
[2024-01-24 16:43:39.48] [Worker 0] New coverage: 1513 instr, 1 contracts, 4 seqs in corpus
[2024-01-24 16:43:39.62] [Worker 0] New coverage: 1513 instr, 1 contracts, 5 seqs in corpus
[2024-01-24 16:43:40.86] [Worker 0] New coverage: 1513 instr, 1 contracts, 6 seqs in corpus
[2024-01-24 16:43:41.96] [status] tests: 0/2, fuzzing: 9848/50000, values: [], cov: 1513, corpus: 6
[2024-01-24 16:43:44.96] [status] tests: 0/2, fuzzing: 20240/50000, values: [], cov: 1513, corpus: 6
[2024-01-24 16:43:47.97] [status] tests: 0/2, fuzzing: 30591/50000, values: [], cov: 1513, corpus: 6
[2024-01-24 16:43:50.97] [status] tests: 0/2, fuzzing: 40422/50000, values: [], cov: 1513, corpus: 6
[2024-01-24 16:43:53.84] [Worker 0] Test limit reached. Stopping.
[2024-01-24 16:43:53.84] [status] tests: 0/2, fuzzing: 50098/50000, values: [], cov: 1513, corpus: 6
echidna_testMarriageSymmetry: passing
echidna_spouseAddressNotZero: passing
```

While running the tests, it can be observed, as mentioned in the initial comment, that new_spouse cannot be zero. However, for symmetry and completeness, I decided to keep the tests and checks done.

Regarding the second issue, the statement `require(new_spouse != address(this));` has been symmetrically added, as done before for the previous issue.

```
function echidna_notSelfMarried() public view returns (bool) {
    return address(Alice) != address(Alice.getSpouse());
}
```

```
[2024-01-25 00:04:13.44] [status] tests: 0/3, fuzzing: 24012/50000, values: [], cov: 1148, corpus: 5
[2024-01-25 00:04:16.44] [status] tests: 0/3, fuzzing: 28014/50000, values: [], cov: 1148, corpus: 5
[2024-01-25 00:04:19.45] [status] tests: 0/3, fuzzing: 34017/50000, values: [], cov: 1148, corpus: 5
[2024-01-25 00:04:22.45] [status] tests: 0/3, fuzzing: 40020/50000, values: [], cov: 1148, corpus: 5
[2024-01-25 00:04:25.45] [status] tests: 0/3, fuzzing: 44022/50000, values: [], cov: 1148, corpus: 5
[2024-01-25 00:04:28.14] [Worker 0] Test limit reached. Stopping.
[2024-01-25 00:04:28.14] [status] tests: 0/3, fuzzing: 51037/50000, values: [], cov: 1148, corpus: 5
echidna_testMarriageSymmetry: passing
echidna_notSelfMarried: passing
echidna_spouseAddressNotZero: passing

Unique instructions: 1148
Unique codehashes: 1
Corpus size: 5
Seed: 7534975614389392119
```

```
34 //parent2 = p2;
35 spouse = address(0);
36 income = 0;
37 tax_allowance = DEFAULT_ALLOWANCE;
38 }
39
40
41 //We require new_spouse != address(0);
42 function marry(address new_spouse) public {
43     // Added checks to see if the two contracts are not already married and if the
44     spouse address is valid
45     require(!isMarried && new_spouse != address(0));
46     require(new_spouse != address(this));
47     Taxpayer sp = Taxpayer(address(new_spouse));
48     require(!sp.getIsMarried());
49     spouse = new_spouse;
50     isMarried = true;
51     sp.setSpouse(address(this));
52     require(marriageSymmetry()); //forse posso evitare, perchè lo metto già in
53     setSpouse
54 }
55
```

2.3 No underage marriage

```
echidna_notSelfMarried: passing
echidna_spouseAddressNotZero: passing
echidna_notUnderage: failed! 🚨
Call sequence:
    setSpouse(0xdeadbeef)

Traces:
```

```
44 require(!isMarried && new_spouse != address(0));
45 require(new_spouse != address(this));
46 Taxpayer sp = Taxpayer(address(new_spouse));
47 require(!sp.getIsMarried());
48 //require(age >= 18 && sp.getAge() >= 18);
49 spouse = new_spouse;
50 isMarried = true;
51 sp.setSpouse(address(this));
52 require(marriageSymmetry());
```



```
echidna_notUnderage: failed! 💣  
Call sequence:  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  haveBirthday()  
  setSpouse(0xdeadbeef)  
  
Traces:
```

This invariant has been checked in an entirely analogous way to the two previous ones. The 'and' condition present in the screenshot provided was indeed split into two and checked separately by the marry() and setSpouse() functions.

```
function echidna_notUnderage() public view returns (bool) {  
    if (Alice.getIsMarried()) {  
        return Alice.getAge() >= 18;  
    }  
    return true;  
}
```

If Alice is married but shes less than 18 years old, there is a violation to report.

```
cov: 1175, corpus: 4  
[2024-01-25 00:24:28.18] [status] tests: 0/4, fuzzing: 33787/50000, values: [],  
cov: 1175, corpus: 4  
[2024-01-25 00:24:31.19] [status] tests: 0/4, fuzzing: 37789/50000, values: [],  
cov: 1175, corpus: 4  
[2024-01-25 00:24:34.19] [status] tests: 0/4, fuzzing: 41791/50000, values: [],  
cov: 1175, corpus: 4  
[2024-01-25 00:24:37.19] [status] tests: 0/4, fuzzing: 47794/50000, values: [],  
cov: 1175, corpus: 4  
[2024-01-25 00:24:39.34] [Worker 0] Test limit reached. Stopping.  
[2024-01-25 00:24:39.34] [status] tests: 0/4, fuzzing: 51796/50000, values: [],  
cov: 1175, corpus: 4  
echidna_testMarriageSymmetry: passing  
echidna_notSelfMarried: passing  
echidna_spouseAddressNotZero: passing  
echidna_notUnderage: passing  
  
Unique instructions: 1175  
Unique codehashes: 1  
Corpus size: 4  
Seed: 1695999081348260065  
  
32  isMarried = false;  
33  //parent1 = p1;  
34  //parent2 = p2;  
35  spouse = address(0);  
36  income = 0;  
37  tax_allowance = DEFAULT_ALLOWANCE;  
38  }  
39  
40  
41  //We require new_spouse != address(0);  
42  function marry(address new_spouse) public {  
43    // Added checks to see if the two contracts are not already married and if the  
    spouse address is valid  
44    require(!isMarried && new_spouse != address(0));  
45    require(new_spouse != address(this));  
46    Taxpayer sp = Taxpayer(address(new_spouse));  
47    require(!sp.getIsMarried());  
48    require(age >= 18 && sp.getAge() >= 18);  
49    spouse = new_spouse;  
50    isMarried = true;  
51    sp.setSpouse(address(this));  
52    require(marriageSymmetry());  
53  }
```



2.4 Divorce

Finally, I want to mention the property that even divorce, with its corresponding function and `resetSpouse()`, must be symmetric. It is not possible to remove the spouse content from only one party. However, there are no specific tests for this property, as it has been directly considered within the scope of the first point of marriage symmetry.

3 Part 2 - Tax allowance

In the second part, for a single person, a tax allowance of 5000 is considered. If two individuals marry each other, they can potentially combine their tax allowances based on their needs using the `transferAllowance()` function, provided that the combined total does not exceed the threshold of 10000. In other words, being two persons, they have two tax allowances available to manage and use, but not more than that. Initially, no checks were provided in the function to transfer allowances, and this resulted in test failures. The initial Echidna test was structured to consider and check both the case of married individuals and unmarried individuals. However, this was later modified with the introduction of a dedicated test function for the latter case. The test function can be observed in figure

```
[2024-01-25 03:18:19.00] [status] tests: 1/5, fuzzing: 48742/50000, values: [],  
cov: 1166, corpus: 8  
[2024-01-25 03:18:19.62] [Worker 0] Test limit reached. Stopping.  
[2024-01-25 03:18:19.62] [status] tests: 1/5, fuzzing: 50055/50000, values: [],  
cov: 1166, corpus: 8  
echidna_taxAllowancePolling: failed! ✨  
  Call sequence:  
    setTaxAllowance(0)  
Traces:  
echidna_testMarriageSymmetry: passing  
echidna_notSelfMarried: passing  
echidna_spouseAddressNotZero: passing  
echidna_notUnderage: passing
```

```
        return true;  
    }  
    function echidna_taxAllowancePolling() public returns (bool) {  
        if (isMarried) {  
            Taxpayer sp = Taxpayer(address(spouse));  
            return tax_allowance + sp.getTaxAllowance() == DEFAULT_ALLOWANCE * 2;  
        }  
        else {  
            return tax_allowance == DEFAULT_ALLOWANCE;  
        }  
    }  
}
```

and follows the logic just described. I modified the functions `setAllowance()`, `divorce()`, `resetSpouse()`, and `transferAllowance()`. To modify the allowance with `setAllowance()`, two people must be married, and the auxiliary function `checkTaxAllowancePooling()` must be called to ensure that the condition remains satisfied. In `transferAllowance()`, I check both that the value to transfer is greater than zero (although, being a uint, this check could be omitted for gas efficiency) and that it is less than the tax allowance from which it is being withdrawn. Finally, when divorcing, the two people must revert to the default with: `tax_allowance = DEFAULT_ALLOWANCE`.



```
[2024-01-25 03:27:55.50] [status] tests: 0/5, fuzzing: 27444/50000, values: [],  
cov: 1177, corpus: 6  
[2024-01-25 03:27:58.50] [status] tests: 0/5, fuzzing: 32463/50000, values: [],  
cov: 1177, corpus: 6  
[2024-01-25 03:28:01.51] [status] tests: 0/5, fuzzing: 38935/50000, values: [],  
cov: 1177, corpus: 6  
[2024-01-25 03:28:04.51] [status] tests: 0/5, fuzzing: 43630/50000, values: [],  
cov: 1177, corpus: 6  
[2024-01-25 03:28:07.40] [Worker 0] Test limit reached. Stopping.  
[2024-01-25 03:28:07.41] [status] tests: 0/5, fuzzing: 50095/50000, values: [],  
cov: 1177, corpus: 6  
echidna_taxAllowancePolling: passing  
echidna_testMarriageSymmetry: passing  
echidna_notSelfMarried: passing  
echidna_spouseAddressNotZero: passing  
echidna_notUnderage: passing  
  
Unique instructions: 1177  
Unique codehashes: 1  
Corpus size: 6  
Seed: 5874019492552647167
```

3.0.1 Single Tax Allowance

A new test has been introduced specifically to check the case of unmarried people, ensuring that `tax_allowance == DEFAULT_ALLOWANCE`.

```
echidna_taxAllowancePolling: passing  
echidna_singleTaxAllowance: passing  
echidna_testMarriageSymmetry: passing  
echidna_notSelfMarried: passing  
echidna_spouseAddressNotZero: passing  
echidna_notUnderage: passing  
  
171 function echidna_singleTaxAllowance() public returns (bool) {  
172     if (!isMarried) {  
173         return tax_allowance == DEFAULT_ALLOWANCE;  
174     }  
175     return true;  
176 }
```

4 Part 3 - Tax allowances for over 65

In the third part, a tax allowance of 7000 has been provided for people aged at least 65, while keeping the other rules unchanged. This means that various cases for marriages must be considered because the desired sum for pooling changes based on the age of the two parties, whether they are above or below 65 years old. To implement these changes, new tests were not constructed; instead, the existing tests for the previous part were modified. The earlier checks were also altered, integrating them with the new ones, aided by the creation of `checkTaxAllowancePooling()`, which mirrors the functionality of the test function, that in the end was modified to directly call this function. Additionally, if a person has shared their tax allowance with their partner and they (or one of them) turn 65, modifying the base allowance, they (or the one above 65) should receive an additional 2000 while still maintaining the previous division. To modify the allowance, a check with an assignment has been introduced in the `haveBirthday()` function:

```
if (age == 65) {  
    tax_allowance = tax_allowance + ALLOWANCELOAP - DEFAULT_ALLOWANCE;  
}
```




With the current conditions, where the contract is initialized with age 0, and the only way to change age is through this function, which adds one year at a time, it is suitable without further interventions. Moreover, there is a factor to consider regarding the tests: despite the problems not initially corrected, the tests would still pass because I had not yet increased the seq-len in the configuration file, i.e., the sequence of actions executed for testing. This meant that, with few available actions, the age never reached the threshold of 65, always falling into the case of the previous part.

```
[2024-01-25 11:47:14.56] [Worker 0] New coverage: 1203 instr, 1 contracts, 8 seq
s in corpus
[2024-01-25 11:47:15.86] [status] tests: 0/6, fuzzing: 29223/50000, values: [],
cov: 1203, corpus: 8
[2024-01-25 11:47:18.87] [status] tests: 0/6, fuzzing: 35156/50000, values: [],
cov: 1203, corpus: 8
[2024-01-25 11:47:21.87] [status] tests: 0/6, fuzzing: 41210/50000, values: [],
cov: 1203, corpus: 8
[2024-01-25 11:47:24.87] [status] tests: 0/6, fuzzing: 47451/50000, values: [],
cov: 1203, corpus: 8
[2024-01-25 11:47:26.15] [Worker 0] New coverage: 1203 instr, 1 contracts, 9 seq
s in corpus
[2024-01-25 11:47:26.15] [Worker 0] Test limit reached. Stopping.
[2024-01-25 11:47:26.15] [status] tests: 0/6, fuzzing: 50051/50000, values: [],
cov: 1203, corpus: 9
echidna_taxAllowancePolling: passing
echidna_singleTaxAllowance: passing
echidna_testMarriageSymmetry: passing
echidna_notSelfMarried: passing
echidna_spouseAddressNotZero: passing
echidna_notUnderage: passing
```

Indeed, after changing this value, with the code already modified but the test conditions not yet updated, they now fail.

```
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
175
176 function echidna_taxAllowancePolling() public returns (bool) {
177
178     if (isMarried) {
179         Taxpayer sp = Taxpayer(address(spouse));
180         return tax_allowance + sp.getTaxAllowance() == DEFAULT_ALLOWANCE * 2;
181     }
182     else {
183         return tax_allowance == DEFAULT_ALLOWANCE;
184     }
185 }
186
187 // può essere tolto, perchè lo faccio già nel primo. Oppure si possono dividere i test in due
188 // per essere più specifici
188 function echidna_singleTaxAllowance() public returns (bool) {
189     if (!isMarried) {
190         return tax_allowance == DEFAULT_ALLOWANCE;
191     }
192     return true;
193 }
194
195
196 /*
197 contract TestTaxpayer is Taxpayer {
Traces:
echidna_singleTaxAllowance: failed! ✨
Call sequence:
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
```



```

pv: 1265, corpus: 4
2024-01-25 12:10:18.62] [status] tests: 1/6, fuzzing: 49127/50000, values: [],
pv: 1265, corpus: 4
2024-01-25 12:10:19.19] [Worker 0] Test limit reached. Stopping.
2024-01-25 12:10:19.19] [status] tests: 1/6, fuzzing: 51128/50000, values: [],
pv: 1265, corpus: 4
chidna_taxAllowancePolling: passing
chidna_singleTaxAllowance: failed: ✨
Call sequence:
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
haveBirthday()
6 function echidna_singleTaxAllowance() public returns (bool) {
7     if (!isMarried) {
8         /*
9         if (age >= 65) {
10            return tax_allowance == ALLOWANCE_OAP;
11        }
12        else {
13            return tax_allowance == DEFAULT_ALLOWANCE;
14        }
15        */
16        return tax_allowance == DEFAULT_ALLOWANCE;
17    }
18    return true;
19 }

```

Then, changing according to the previously explained logic:

```

function checkTaxAllowancePolling() public view returns (bool) {

    if (isMarried) {
        Taxpayer sp = Taxpayer(address(spouse));
        if (age >= 65) {
            if (sp.getAge() >= 65) {
                return tax_allowance + sp.getTaxAllowance() ==
                    ALLOWANCEOAP * 2;
            }
            else {
                return tax_allowance + sp.getTaxAllowance() ==
                    ALLOWANCEOAP + DEFAULT_ALLOWANCE;
            }
        }
        else { // The first partner is under 65 years old.
            if (sp.getAge() >= 65) { // ma il secondo no
                return tax_allowance + sp.getTaxAllowance() ==
                    DEFAULT_ALLOWANCE + ALLOWANCEOAP;
            }
            else { // Both are under 65 years old.
                return tax_allowance + sp.getTaxAllowance() ==
                    DEFAULT_ALLOWANCE * 2;
            }
        }
    }
    return true;
}

function echidna_taxAllowancePolling() public view returns (bool) {
    return Alice.checkTaxAllowancePolling();
}

```



```
cov: 1265, corpus: 4
[2024-01-25 12:16:13.25] [status] tests: 0/6, fuzzing: 49438/50000, values: [],
cov: 1265, corpus: 4
[2024-01-25 12:16:13.61] [Worker 0] Test limit reached. Stopping.
[2024-01-25 12:16:13.61] [status] tests: 0/6, fuzzing: 50956/50000, values: [],
cov: 1265, corpus: 4
echidna_taxAllowancePolling: passing
echidna_singleTaxAllowance: passing
echidna_testMarriageSymmetry: passing
echidna_notSelfMarried: passing
echidna_spouseAddressNotZero: passing
echidna_notUnderage: passing

function echidna_singleTaxAllowance() public returns (bool) {
    if (isMarried) {
        if (age >= 65) {
            return tax_allowance == ALLOWANCE_OAP;
        }
        else {
            return tax_allowance == DEFAULT_ALLOWANCE;
        }
    }
    return tax_allowance == DEFAULT_ALLOWANCE;
}
return true;
```

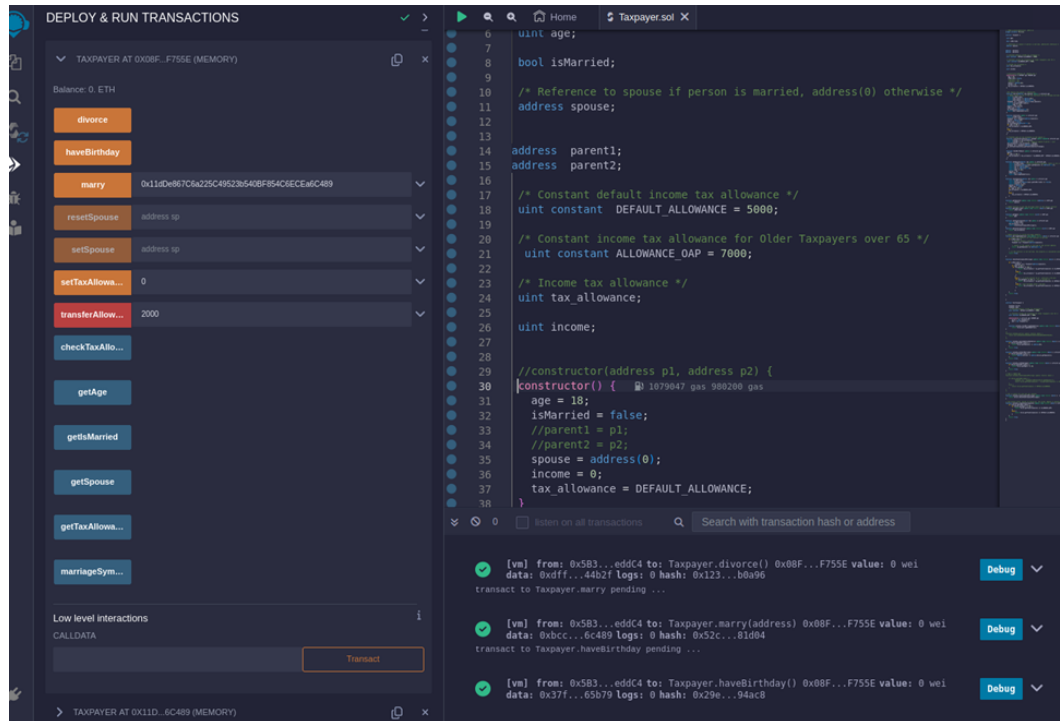
5 Part 3 - Remix IDE and Conclusion

Testing the code on Remix IDE allowed me to address warnings related to the appropriate visibility (*view*) for certain functions (both getters and non-getters) and correct a code error involving unreachable code caused by an incorrect repetition of a *return*.

```
function echidna_singleTaxAllowance() public returns (bool) {
    if (!isMarried) {
        if (age >= 65) {
            return tax_allowance == ALLOWANCE_OAP;
        }
        else {
            return tax_allowance == DEFAULT_ALLOWANCE;
        }
        return tax_allowance == DEFAULT_ALLOWANCE;
    }
    return true;
}
```

The screenshot shows the Remix IDE interface. On the left, the 'Solidity compiler' panel displays three warnings: 'Warning: Function state mutability can be restricted to view' for functions 'getSpouse' (line 107), 'getIsMarried' (line 112), and 'getAge' (line 116). The main editor shows the Solidity code for 'Taxpayer.sol', including functions like 'getSpouse', 'getIsMarried', 'getAge', 'setTaxAllowance', 'getTaxAllowance', and 'marriageSymmetry'. The right sidebar shows the 'ABI' and 'Bytecode' tabs.

I was able to manually test the functions to gain a better understanding of how they work. Being able to test them individually was also very useful for better identifying issues in some cases. Some tests were conducted by manually setting certain values during initialization, such as starting the age at 18, as seen in the screenshot, or using specific addresses. In the screenshot, it's also visible that, as explained earlier, I have left *parent1* and *parent2* commented out for now.



Some notes:

- Since the moment the screenshot was taken, parts of the code, comments, and the configuration file have been modified incrementally.
- It was not possible to display the classic tabular output on the screen due to terminal limitations, so it was chosen to print it in minimal format using the `-format` option.
- Some *require* statements could be omitted for gas efficiency. They have been either kept (usually with a comment nearby) or left commented out (so as comments) for completeness.